

Balancing Accuracy and Sustainability: Energy-Efficient Transformer Models for Scientific Text Classification

Abstract

This study proposes a resource-efficient framework for multi-class classification of scientific texts while preserving competitive accuracy. We fine-tune four widely used Transformer models—BERT, RoBERTa, ALBERT, and DistilBERT—on a dataset of 50,680 ScienceDaily articles across three domains: Health Sciences, Physical Sciences & Technology, and Environmental Sciences. To enhance efficiency, we apply three compression techniques individually: Dynamic Batching, Adapter Modules, and Genetic Algorithms. Two lightweight models, Electra-small and MobileBERT, are additionally evaluated as comparative baselines. Experimental results demonstrate that our approach achieves up to 87.5% reduction in energy consumption while maintaining performance levels close to the uncompressed models. Energy usage and carbon emissions are systematically measured with CodeCarbon, ensuring transparent evaluation. This work highlights practical strategies for sustainable NLP, offering scalable methods to balance performance and environmental responsibility in scientific text classification.

Keywords: Transformer models, multi-class classification, compression techniques, energy efficiency, sustainable computing

1. Introduction

Transformer models have revolutionized natural language processing (NLP), enabling energy-aware applications in scientific research, healthcare, and environmental monitoring Vaswani et al. (2017). Their ability to capture complex contextual relationships has made them indispensable for multi-class text classification, such as categorizing scientific literature or analyzing environmental data. However, their computational intensity raises significant

sustainability concerns, with training a single model consuming energy equivalent to substantial carbon emissions. For example, Paula et al. showed that optimizing Transformer architectures can reduce energy use by up to 50% while preserving performance, highlighting the potential for sustainable computing solutions [Paula et al. \(2025\)](#). As NLP applications scale, minimizing the environmental footprint of Transformers is essential to align with goals of power-efficient computing and ecological impact reduction.

Prior research on Transformers has primarily focused on accuracy or inference speed, often neglecting energy efficiency and thermal management [Ganesh et al. \(2021\)](#). Existing studies on model compression, such as pruning, quantization, and knowledge distillation, typically evaluate these techniques in isolation or on simpler tasks like binary classification, leaving their impact on multi-class scientific text classification underexplored [Ganesh et al. \(2021\)](#). Moreover, comprehensive energy tracking using tools like CodeCarbon is rarely applied, limiting insights into power-aware resource management. This gap hinders the deployment of Transformers in sustainable computing environments, such as edge devices or green data centers, necessitating innovative approaches to eco-efficient neural computing.

We propose an eco-efficient Transformer framework that employs Genetic Algorithms (GA), Adapter Modules, and Dynamic Batching to optimize multi-class text classification on a dataset of 50,680 ScienceDaily articles, categorized into Health Sciences, Physical Sciences & Technology, and Environmental Sciences. GA optimizes model architectures by pruning redundant components, Adapter Modules enable lightweight fine-tuning, and Dynamic Batching adapts resource allocation to workload dynamics. Inspired by Paula et al.'s work on combining compression techniques for energy-efficient AI [Paula et al. \(2025\)](#), our framework synergistically combines these methods to reduce energy consumption by up to 87.5% while maintaining near-baseline accuracy. Energy and carbon emissions are rigorously tracked using CodeCarbon, ensuring a precise assessment of environmental impact [Courty et al. \(2024\)](#).

This study introduces a novel framework that employs Genetic Algorithms, Adapter Modules, and Dynamic Batching to optimize Transformer models for both performance and energy efficiency in multi-class text classification, achieving up to 87.5% reduction in energy consumption while maintaining near-baseline accuracy on a dataset of 50,680 ScienceDaily articles. It provides a comprehensive evaluation of four Transformer models—BERT, RoBERTa, ALBERT, and DistilBERT—on this large-scale scientific dataset,

benchmarking their performance and energy efficiency against lightweight architectures like Electra-small and MobileBERT to identify optimal configurations for sustainable AI. Additionally, the study offers practical guidelines for deploying eco-efficient models in resource-constrained environments, advancing Green AI research and enabling scalable neural computing applications with reduced environmental impact.

The rest of this paper is structured as follows: Section 2 reviews related work on Transformer models, compression techniques, and sustainable AI. Section 3 details the methodology, including dataset preprocessing, architecture of models and optimization techniques. Section 4 presents experimental results, model configurations, analyzing performance and energy trade-offs. Section 5 concludes with implications and future directions for sustainable NLP.

2. Related Work

2.1. Transformer Models in Text Classification

Transformer models have revolutionized text classification through self-attention mechanisms that capture complex contextual relationships Vaswani et al. (2017). BERT leverages bidirectional pre-training with Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) for robust performance in multi-class text classification tasks Devlin et al. (2019). RoBERTa enhances BERT by eliminating NSP, using dynamic masking, and training on larger datasets, leading to superior generalization in classification benchmarks like GLUE Liu et al. (2019); Wang et al. (2018). ALBERT reduces parameters via factorized embeddings and cross-layer sharing, maintaining high accuracy in classification while being more efficient Lan et al. (2020). DistilBERT employs knowledge distillation to achieve 97% of BERT’s classification performance with 40% fewer parameters Sanh et al. (2019). Electra uses a generator-discriminator framework for efficient pre-training, improving results in classification tasks Clark et al. (2020), and MobileBERT optimizes for low-latency inference on resource-constrained devices, making it suitable for mobile classification applications Sun et al. (2020). These models, while powerful for classification, demand significant computational resources, necessitating sustainable optimization strategies to minimize energy consumption and environmental impact.

2.2. Optimization for Energy Efficiency

Optimization techniques reduce the computational footprint of Transformers. Pruning eliminates low-impact weights or layers, reducing parameters by up to 90% in models like BERT with minimal accuracy loss [Ganesh et al. \(2021\)](#). Knowledge distillation transfers knowledge from a large model to a smaller one, as seen in DistilBERT and TinyBERT [Sanh et al. \(2019\)](#); [Jiao et al. \(2020\)](#). Quantization lowers numerical precision to decrease memory and inference costs [Wu et al. \(2020\)](#). Adapter Modules enable parameter-efficient fine-tuning by adding small, trainable layers, reducing memory usage by up to 80% [Houlsby et al. \(2019\)](#); [Pfeiffer et al. \(2020\)](#). Dynamic Batching optimizes hardware utilization by adjusting batch sizes, reducing training times and energy demands [Ye et al. \(2020\)](#). Paula et al. demonstrated that targeted optimizations can cut Transformer energy use by 50%, emphasizing the value of efficiency-focused strategies [Paula et al. \(2025\)](#). However, these techniques are often applied individually, and their impact on multi-class classification requires further exploration.

2.3. Sustainable Computing in AI

Sustainable computing in AI aims to minimize environmental impact through energy-aware and thermal-aware management [Schwartz et al. \(2020\)](#). Paula et al. applied pruning, knowledge distillation, and quantization to achieve energy savings in NLP tasks, highlighting their role in sustainable AI [Paula et al. \(2025\)](#). Tools like CodeCarbon provide precise tracking of energy consumption and carbon emissions, supporting eco-friendly design [Courty et al. \(2024\)](#). These advancements enable scalable AI in resource-constrained settings, aligning with sustainable computing principles for applications with ecological and societal impacts.

2.4. Gaps and Research Opportunities

While model optimization has advanced, studies often focus on binary classification or small datasets, limiting applicability to complex tasks like multi-class scientific text classification [Zhang et al. \(2015\)](#); [Ganesh et al. \(2021\)](#). Few works apply multiple optimization techniques together or use comprehensive energy tracking. Paula et al.'s framework, while effective, does not incorporate adaptive strategies like Dynamic Batching [Paula et al. \(2025\)](#). Our work addresses these gaps by evaluating four Transformer models—BERT, RoBERTa, ALBERT, and DistilBERT—with GA, Adapter

Modules, and Dynamic Batching on a large-scale scientific dataset, offering insights into sustainable computing for NLP. We benchmark against lightweight baselines like Electra-small and MobileBERT, providing practical guidelines for energy-efficient AI deployment.

3. Methodology

3.1. Data Collection and Preprocessing

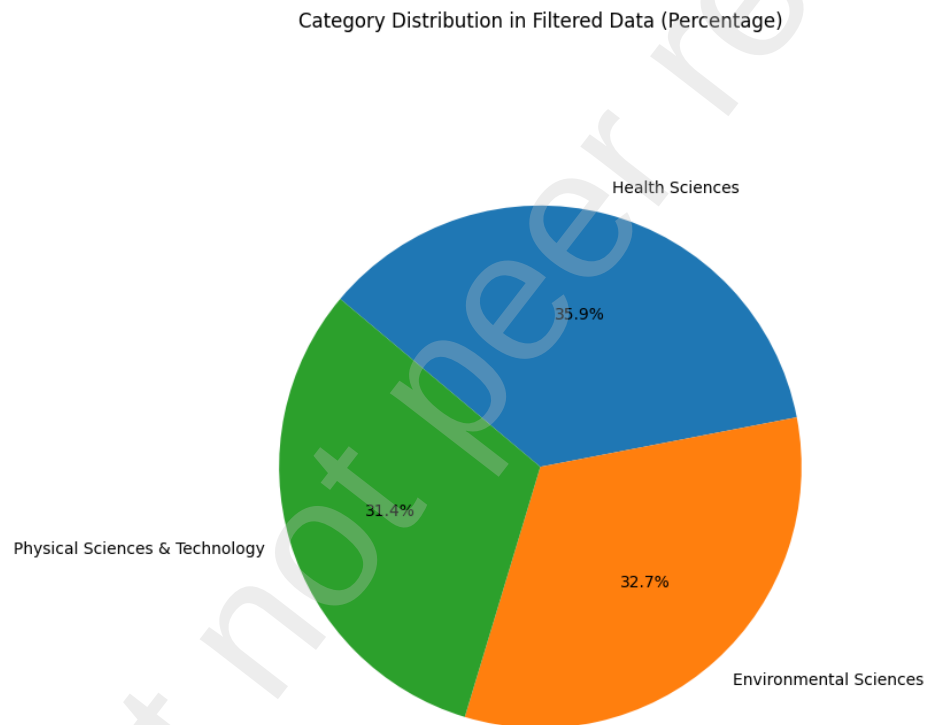


Figure 1: Category Distribution in Filtered Data (Percentage)

The dataset was collected by scraping approximately 75,000 ScienceDaily articles, including URL, title, publication date, source, category, summary,

and content. We selected three categories—Health Sciences, Physical Sciences & Technology, and Environmental Sciences—using titles and summaries as input features. Preprocessing used Python (Pandas, scikit-learn) and the BART tokenizer Lewis et al. (2020) for tokenization, filtering by token counts and semantic similarity (cosine similarity ≥ 0.2 via TF-IDF vectors). Articles were filtered to titles with 8–32 tokens and summaries with 12–128 tokens, yielding 50,680 articles. Stratification ensured balance across categories and vocabulary overlap (tokens with frequency ≥ 3 , binned into 5 quantiles). Category distribution is balanced (see Figure 1). Stratified splitting was performed into train (80%, 40,544 samples), validation (10%, 5,068 samples), and test (10%, 5,068 samples) sets with random seed 42.

Table 1: Token Statistics

Statistic	Title Tokens	Summary Tokens
Mean	14.47	57.87
Standard Deviation	4.09	24.07
Minimum	4	9
Maximum	44	253
25% Quartile	12	39
50% Quartile	14	54
75% Quartile	17	74

Table 2: Dataset Split

Dataset Split	Sample Count
Train (80%)	40,544
Validation (10%)	5,068
Test (10%)	5,068

3.2. Baseline Classification Models

This section evaluates six Transformer models as baselines for multi-class text classification: BERT, RoBERTa, ALBERT, DistilBERT, Electra-small, and MobileBERT. These models were selected for their state-of-the-art performance in NLP tasks and varying architectural efficiencies, providing a comprehensive benchmark for energy consumption in classification.

BERT (Bidirectional Encoder Representations from Transformers) pre-trains deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers [Devlin et al. \(2019\)](#). It employs two pre-training objectives: (1) Masked Language Modeling (MLM), where 15% of tokens are randomly masked (80% replaced with [MASK], 10% with a random token, 10% unchanged), and the model predicts the original tokens based on context; (2) Next Sentence Prediction (NSP), a binary classification task predicting if two sentences are consecutive. The architecture consists of stacked Transformer encoder layers, each with self-attention and feed-forward components. We use `bert-base-uncased`, with 12 layers ($L = 12$), hidden size of 768 ($H = 768$), 12 self-attention heads ($A = 12$), and approximately 110 million parameters. The model output for classification is derived from the [CLS] token’s final hidden state, passed through a linear layer and softmax:

$$P(y | x) = \text{softmax}(W \cdot h_{[\text{CLS}]} + b), \quad (1)$$

where $h_{[\text{CLS}]} \in \mathbb{R}^{768}$ is the [CLS] token embedding, $W \in \mathbb{R}^{C \times 768}$ (with $C = 3$ classes), and b is the bias. This setup excels in capturing semantic relationships for multi-class tasks.

RoBERTa (Robustly Optimized BERT Approach) improves BERT by optimizing its pre-training [Liu et al. \(2019\)](#). It eliminates NSP, uses dynamic masking (masks vary per epoch), trains on larger datasets (160GB vs. BERT’s 16GB), and employs larger batch sizes (up to 8K). This enhances generalization and performance on downstream tasks. The `roberta-base` model has 12 layers, hidden size 768, 12 attention heads, and 125 million parameters. Its architecture mirrors BERT’s but benefits from extended training (e.g., 500K steps). For classification, RoBERTa uses the same [CLS]-based mechanism, fine-tuned with a cross-entropy loss:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}), \quad (2)$$

where $y_{i,c}$ is the true label and $\hat{y}_{i,c}$ is the predicted probability for sample i and class c . RoBERTa’s robust pre-training makes it particularly effective for complex classification tasks.

ALBERT (A Lite BERT) reduces BERT’s parameter count through factorized embedding parameterization and cross-layer parameter sharing [Lan](#)

et al. (2020). In factorized embeddings, the vocabulary embedding size E (e.g., 128) is smaller than the hidden size H (768), reducing parameters from $O(V \cdot H)$ to $O(V \cdot E + E \cdot H)$, where V is the vocabulary size. Cross-layer sharing reuses parameters across all layers, reducing parameters from $O(L \cdot H^2)$ to $O(H^2)$. ALBERT uses Sentence Order Prediction (SOP) instead of NSP, a binary task distinguishing correct vs. swapped sentence orders, improving inter-sentence coherence. We use `albert-base-v2`, with 12 effective layers (shared parameters), hidden size 768, 12 attention heads, and approximately 12 million parameters. For classification, ALBERT processes inputs similarly to BERT, with the output layer:

$$P(y | x) = \text{softmax}(W \cdot h_{[\text{CLS}]} + b). \quad (3)$$

Its lightweight design reduces memory and training time, ideal for sustainable NLP.

DistilBERT distills BERT’s knowledge into a smaller model via a student-teacher framework Sanh et al. (2019). It uses a triple loss during pre-training: MLM loss (like BERT), distillation loss (soft cross-entropy matching BERT’s output probabilities), and cosine embedding loss aligning hidden states. Architecturally, it halves BERT’s layers, removes token-type embeddings and the pooler, yielding `distilbert-base-uncased` with 6 layers, hidden size 768, 12 attention heads, and approximately 66 million parameters. The classification head is:

$$P(y | x) = \text{softmax}(W \cdot \text{DistilBERT}(x)_{[\text{CLS}]} + b). \quad (4)$$

DistilBERT achieves 60% faster inference and 40% fewer parameters while retaining 97% of BERT’s performance, making it efficient for resource-constrained settings.

Electra-small (Efficiently Learning an Encoder that Classifies Token Replacements Accurately) uses a novel pre-training approach with a generator-discriminator framework Clark et al. (2020). The generator, a small Transformer, performs MLM by predicting masked tokens, while the discriminator (Electra) predicts whether each token is original or replaced by the generator. This replaced token detection task trains the discriminator on all input tokens, improving efficiency over MLM. We use `google/electra-small-discriminator`, with 12 layers ($L = 12$), hidden size 256 ($H = 256$), 4 attention heads ($A = 4$), and approximately 14 million parameters. For classification, Electra-small uses the [CLS] token’s final

hidden state, processed similarly to BERT:

$$P(y | x) = \text{softmax}(W \cdot \text{Electra}(x)_{[\text{CLS}]} + b). \quad (5)$$

Its small size and efficient pre-training make it suitable for resource-constrained environments, offering a balance between performance and computational cost.

MobileBERT is a compact Transformer model designed for mobile and edge devices, optimizing BERT for low-latency inference [Sun et al. \(2020\)](#). It uses bottleneck layers, layer normalization optimizations, and knowledge distillation from a larger BERT teacher. MobileBERT reduces parameters through factorized embeddings and progressive knowledge transfer, maintaining performance with fewer resources. We use `google/mobilebert-uncased`, with 24 layers ($L = 24$), hidden size 512 ($H = 512$), 4 attention heads ($A = 4$), and approximately 25 million parameters. The classification head is:

$$P(y | x) = \text{softmax}(W \cdot \text{MobileBERT}(x)_{[\text{CLS}]} + b). \quad (6)$$

MobileBERT’s design prioritizes low computational overhead, making it ideal for sustainable NLP applications.

3.3. Compression Techniques

To reduce the computational and energy costs of Transformer models while maintaining classification accuracy, we apply three compression techniques: Dynamic Batching, Adapter Modules, and Genetic Algorithm (GA). Each method targets either model architecture or training dynamics to minimize resource consumption, aligning with the goal of sustainable NLP.

Dynamic Batching adaptively adjusts the batch size during training to optimize hardware utilization and reduce idle time [Ye et al. \(2020\)](#). Fixed batch sizes (e.g., 32) can lead to underutilization or memory errors. Dynamic Batching monitors training metrics, such as gradient variance or validation loss, to scale batch sizes dynamically. For example, it starts with a small batch size (e.g., $B_0 = 16$) for stability in early epochs and increases to a maximum (e.g., $B_{\max} = 128$) as gradients stabilize. The batch size update at training step t is governed by:

$$B_t = \min(B_{\max}, B_{t-1} \cdot (1 + \eta \cdot \text{sign}(\Delta \mathcal{L}_v))), \quad (7)$$

where B_t is the batch size at step t , η is a scaling factor (set to 0.1), $\Delta \mathcal{L}_v = \mathcal{L}_v(t) - \mathcal{L}_v(t-1)$ is the change in validation loss, and $B_{\max}$ caps the

size to prevent memory overflow. If $\Delta\mathcal{L}_v > 0$ (loss increases), B_t decreases to introduce stochasticity; if $\Delta\mathcal{L}_v < 0$, B_t increases to improve throughput. This is implemented by integrating system resource monitoring (e.g., GPU memory usage via PyTorch) and adjusting B_t per epoch or after a fixed number of steps (e.g., 100). Dynamic Batching reduces training time by processing more samples efficiently per iteration, lowering energy consumption (measured in kWh via CodeCarbon) while maintaining convergence.

Adapter Modules enable parameter-efficient fine-tuning by inserting small, trainable bottleneck layers into each Transformer layer while freezing the pre-trained weights [Houlsby et al. \(2019\)](#). An adapter consists of a down-projection matrix $W_{\text{down}} \in \mathbb{R}^{h \times r}$ (where $h = 768$ is the hidden size, $r = 16$ is the bottleneck dimension), a ReLU activation σ , and an up-projection matrix $W_{\text{up}} \in \mathbb{R}^{r \times h}$. For an input hidden state $h \in \mathbb{R}^h$, the adapter output is computed as:

$$h' = h + W_{\text{up}} \cdot \sigma(W_{\text{down}} \cdot h), \quad (8)$$

and added as a residual connection after the self-attention or feed-forward sublayer. The number of parameters per adapter is $2 \cdot h \cdot r + r$, adding approximately 0.5–1% of the model’s parameters (e.g., 1M for BERT’s 110M). Implemented via the Parameter-Efficient Fine-Tuning (PEFT) library, adapters are inserted into each of the L layers (e.g., $L = 12$ for BERT), and only these parameters are updated during fine-tuning. The loss function remains the cross-entropy loss (Equation 2). By limiting gradient computations to adapters, this method reduces memory usage and training time, lowering energy consumption while preserving near-baseline accuracy.

Genetic Algorithm (GA) optimizes model architectures through evolutionary principles, including selection, crossover, and mutation [Reinikainen \(2024\)](#). GA treats hyperparameters as a genome, such as the number of Transformer layers $L \in [6, 12]$ and hidden size $H \in [384, 768]$ (in multiples of 12 for compatibility with attention heads). The algorithm initializes a population of $N_p = 50$ candidate models, each with a configuration (L_i, H_i) . The fitness function evaluates the trade-off between performance and efficiency:

$$f = \alpha(1 - \text{accuracy}) + \beta \frac{\text{parameters}}{\text{max_parameters}} + \gamma \frac{\text{training_time}}{\text{max_time}}, \quad (9)$$

where $\alpha = 0.6$, $\beta = 0.3$, $\gamma = 0.1$ balance accuracy loss, parameter count (relative to the original model’s maximum, e.g., 110M for BERT), and training time (normalized by the baseline’s maximum time). The GA process is:

1. **Initialization:** Generate N_p random configurations within bounds (e.g., $L \in [6, 12]$, $H \in [384, 768]$).
2. **Evaluation:** Train each model on a subset of the training data (e.g., 10% for speed) and compute f using validation accuracy, parameter count, and training time.
3. **Selection:** Select the top $k = 20$ models based on f using tournament selection.
4. **Crossover:** Pair selected models and combine their L and H (e.g., average or swap values) with probability $p_c = 0.8$.
5. **Mutation:** Randomly alter L or H (e.g., increment/decrement by 1 or 12) with probability $p_m = 0.1$.
6. **Iteration:** Repeat for $G = 50$ generations or until f converges.

The best configuration (e.g., $L = 8$, $H = 512$) is fine-tuned fully. By reducing parameters (e.g., from $O(L \cdot H^2)$ to $O(L' \cdot H'^2)$ where $L' < L$, $H' < H$), GA lowers computational complexity, reducing energy consumption as fewer operations decrease power draw, measured via CodeCarbon.

All techniques are implemented in Python using PyTorch and Hugging Face Transformers. Energy consumption is tracked during fine-tuning to quantify savings, ensuring alignment with green AI principles.

3.4. Energy Measurement Framework

Energy measurement is critical for assessing the environmental impact of Transformer models. We use CodeCarbon [Courty et al. \(2024\)](#), an open-source tool that tracks energy consumption (kWh) and carbon emissions (kgCO₂e) across CPU, RAM, and cooling systems. CodeCarbon integrates hardware-specific power draw data with region-specific emission factors (e.g., kgCO₂e/kWh based on local grid mix). The total energy consumption is:

$$E_{\text{total}} = E_{\text{CPU}} + E_{\text{RAM}}, \quad (10)$$

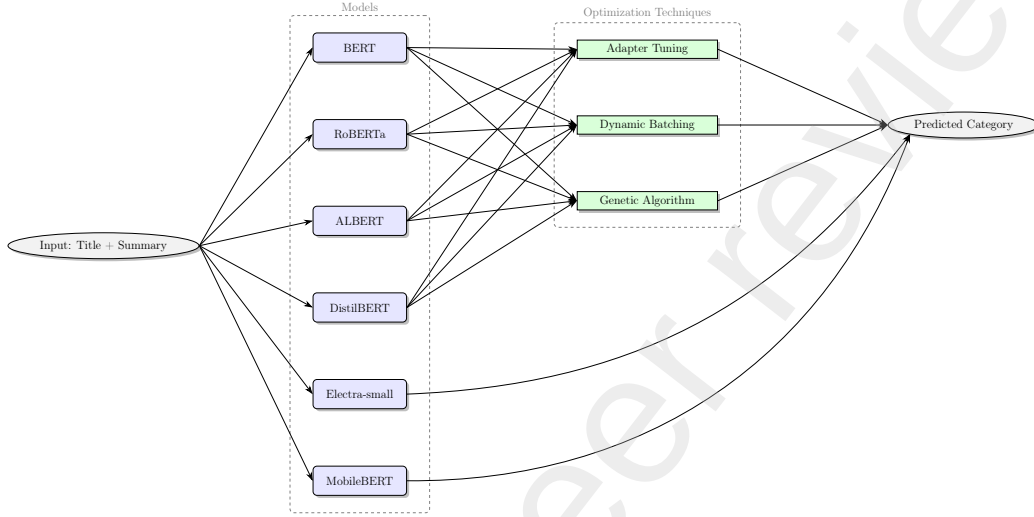
where each component's energy is measured via system-level monitoring. Emissions are calculated as:

$$\text{CO}_2\text{e} = E_{\text{total}} \cdot \text{EF}, \quad (11)$$

where EF is the emission factor. We configure CodeCarbon to track fine-tuning and inference phases, aggregating multiple runs for reliability. This framework quantifies the impact of Dynamic Batching, Adapters, and GA, enabling comparisons of energy savings and supporting green AI principles [Schwartz et al. \(2020\)](#).

3.5. Overall Workflow

Figure 2: Overall Architecture of Models and Optimization Techniques



The workflow integrates data preprocessing, model fine-tuning, optimization, energy measurement, and performance evaluation into a streamlined pipeline for sustainable multi-class text classification, as shown in Figure 3. Approximately 75,000 ScienceDaily articles are preprocessed and filtered to 50,680, then split into training (80%, 40,544 samples), validation (10%, 5,068 samples), and test (10%, 5,068 samples) sets. Four Transformer models—BERT, RoBERTa, ALBERT, and DistilBERT—are fine-tuned on the training set using the Hugging Face Transformers library with an AdamW optimizer and cross-entropy loss. To enhance efficiency, optimization techniques—Dynamic Batching, Adapter Modules, and Genetic Algorithm—reduce computational costs while maintaining accuracy. Energy consumption (kWh) and carbon emissions (kgCO₂e) are tracked during fine-tuning and inference using CodeCarbon. Finally, models are evaluated on the test set using accuracy, precision, recall, and F1-score to assess performance and efficiency trade-offs, promoting eco-friendly NLP practices.

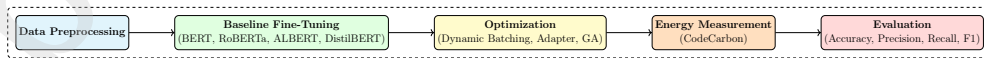


Figure 3: Integrated Pipeline for Sustainable NLP

4. Experimental Results

This section presents the performance and energy efficiency results of the multi-class text classification pipeline, evaluating six Transformer models: BERT, RoBERTa, ALBERT, DistilBERT, Electra-small, and MobileBERT. As described in Section 3.2, BERT, RoBERTa, ALBERT, and DistilBERT are tested in their baseline configurations and with three optimization techniques: Dynamic Batching, Adapter Modules, and Genetic Algorithm (GA). Electra-small and MobileBERT are evaluated only in their baseline configurations to provide additional benchmarks for energy-efficient models. Performance is assessed using accuracy, precision, recall, and F1-score on the test set of 5,068 samples, while energy efficiency is measured in terms of total energy consumption (kWh), CO₂ emissions (kgCO₂e), CPU energy, RAM energy, training time (minutes), and energy per sample (kWh/sample), with a visual comparison of accuracy versus energy consumption in Figure 4.

4.1. Configuration

4.1.1. Training Parameters

The model was fine-tuned using hyperparameters derived from the comparative study by Paula et al. (2025), which demonstrated this configuration as the optimal set for BERT-based models in similar text classification tasks.

- Batch size: 16
- Learning rate: 2×10^{-5}
- Number of epochs: 3

These hyperparameters were adopted based on the findings in Paula et al. (2025), where they were rigorously tested and proven to provide the best balance of performance, convergence, and efficiency. The learning rate of 2×10^{-5} ensures stable optimization without overshooting minima, the 3 epochs allow adequate training while conserving computational resources, and the batch size of 16 optimizes GPU memory usage for consistent gradient updates. This setup was exclusively utilized to achieve high accuracy with minimal costs.

4.1.2. Hardware configuration

The experiments were carried out on a virtual machine configured with the following specifications:

- Operating System: Linux-6.6.56+-x86_64-with-glibc2.35
- CPU: Intel(R) Xeon(R) CPU @ 2.00GHz
- GPU: Tesla P100-PCIE-16GB
- RAM: 31.35 GB

4.2. Performance Results for BERT Models

Table 3 summarizes the performance and energy metrics for BERT configurations. The baseline BERT model achieves high accuracy (0.9280) but incurs significant energy costs, consuming 0.1421 kWh and emitting 0.0406 kgCO₂e. Among the optimized configurations, BERT-Dynamic stands out, maintaining strong accuracy (0.9232, a drop of only 0.0048) while reducing total energy by a factor of 8 (to 0.0178 kWh), CO₂ emissions by a factor of 16.4 (to 0.00247 kgCO₂e), and training time to 17.1 minutes. BERT with Adapters offers a balanced approach, with accuracy close to the baseline (0.9256) and total energy reduced by a factor of 1.3 (to 0.1065 kWh). BERT GA Optimized prioritizes efficiency, achieving substantial energy reduction by a factor of 5.9 (to 0.0240 kWh) but with a more noticeable accuracy drop (0.9092). These results highlight a trade-off where energy and time decrease dramatically with GA, but Dynamic Batching is preferable when accuracy retention is critical.

Metric	BERT (Baseline)	BERT-Dynamic	BERT with Adapters	BERT GA Optimized
Accuracy	0.9280	0.9232	0.9256	0.9092
Precision	0.9281	0.9233	0.9258	0.9092
Recall	0.9280	0.9232	0.9256	0.9092
F1-Score	0.9280	0.9233	0.9257	0.9093
Total Energy (kWh)	0.1421	0.0178	0.1065	0.0240
CO ₂ (kgCO ₂ e)	0.0406	0.00247	0.0148	0.00643
CPU (kWh)	0.0967	0.0121	0.0724	0.0163
RAM (kWh)	0.0455	0.00569	0.0341	0.00769
Time (min)	136.5	17.1	102.3	23.1
Energy/Sample (kWh/sample)	3.51×10^{-6}	4.39×10^{-7}	2.63×10^{-6}	5.93×10^{-7}

Table 3: Performance and Energy Metrics for BERT Configurations

4.3. Energy Efficiency Results for Related Models

We extend the evaluation to RoBERTa, DistilBERT, and ALBERT, with full metrics in Tables 4, 5, and 6. RoBERTa’s baseline achieves high accuracy (0.9325) but consumes 0.1402 kWh and emits 0.0769 kgCO₂e. RoBERTa-Dynamic balances performance (accuracy 0.9286) with significant efficiency gains, reducing total energy by a factor of 5.8 (to 0.0240 kWh) and CO₂ emissions by a factor of 11.2 (to 0.00685 kgCO₂e). RoBERTa with Adapters maintains near-baseline accuracy (0.9280) with total energy reduced by a factor of 1.3 (to 0.1051 kWh).

Metric	RoBERTa (Baseline)	RoBERTa-Dynamic	RoBERTa with Adapters	RoBERTa GA Optimized
Accuracy	0.9325	0.9286	0.9280	0.9039
Precision	0.9328	0.9288	0.9281	0.9039
Recall	0.9325	0.9286	0.9280	0.9039
F1-Score	0.9326	0.9286	0.9280	0.9039
Total Energy (kWh)	0.1402	0.0240	0.1051	0.0219
CO ₂ (kgCO ₂ e)	0.0769	0.00685	0.0146	0.00304
CPU (kWh)	0.0953	0.0163	0.0715	0.0149
RAM (kWh)	0.0448	0.00768	0.0336	0.00701
Time (min)	134.6	23.1	100.9	21.0
Energy/Sample (kWh/sample)	3.46×10^{-6}	5.92×10^{-7}	2.59×10^{-6}	5.40×10^{-7}

Table 4: Performance and Energy Metrics for RoBERTa Configurations

DistilBERT’s baseline offers solid accuracy (0.9260) with lower energy (0.0732 kWh) than larger models. DistilBERT-Dynamic excels, slightly improving accuracy (0.9288) while reducing total energy by a factor of 7.9 (to 0.00929 kWh) and CO₂ emissions by a factor of 16.2 (to 0.00129 kgCO₂e). DistilBERT with Adapters reduces total energy by a factor of 1.4 (to 0.0521 kWh) with minimal accuracy loss (0.9246).

Metric	DistilBERT (Baseline)	DistilBERT-Dynamic	DistilBERT with Adapters	DistilBERT GA Optimized
Accuracy	0.9260	0.9288	0.9246	0.9120
Precision	0.9264	0.9290	0.9248	0.9124
Recall	0.9260	0.9288	0.9246	0.9120
F1-Score	0.9261	0.9289	0.9247	0.9121
Total Energy (kWh)	0.0732	0.00929	0.0521	0.0151
CO ₂ (kgCO ₂ e)	0.0209	0.00129	0.00723	0.00967
CPU (kWh)	0.0498	0.00632	0.0354	0.0102
RAM (kWh)	0.0234	0.00297	0.0167	0.00482
Time (min)	70.3	8.92	50.1	14.5
Energy/Sample (kWh/sample)	1.81×10^{-6}	2.29×10^{-7}	1.29×10^{-6}	3.71×10^{-7}

Table 5: Performance and Energy Metrics for DistilBERT Configurations

ALBERT’s baseline achieves accuracy of 0.9266 with 0.1286 kWh. ALBERT-Dynamic slightly improves accuracy (0.9276) and reduces total energy by a factor of 6.9 (to 0.0186 kWh) and CO₂ emissions by a factor of 3.6 (to 0.00497 kgCO₂e). ALBERT with Adapters reduces total energy by a factor of 1.3

(to 0.0989 kWh) but shows higher CO₂ emissions (0.0635 kgCO₂e vs. baseline 0.0178 kgCO₂e), likely due to the computational overhead of training additional adapter parameters.

Metric	ALBERT (Baseline)	ALBERT-Dynamic	ALBERT with Adapters	ALBERT GA Optimized
Accuracy	0.9266	0.9276	0.9225	0.9041
Precision	0.9266	0.9277	0.9226	0.9042
Recall	0.9266	0.9276	0.9225	0.9041
F1-Score	0.9266	0.9276	0.9225	0.9042
Total Energy (kWh)	0.1286	0.0186	0.0989	0.0236
CO ₂ (kgCO ₂ e)	0.0178	0.00497	0.0635	0.01295
CPU (kWh)	0.0874	0.0126	0.0672	0.0161
RAM (kWh)	0.0411	0.00595	0.0316	0.00755
Time (min)	123.5	17.8	94.9	22.7
Energy/Sample (kWh/sample)	3.17×10^{-6}	4.58×10^{-7}	2.44×10^{-6}	5.82×10^{-7}

Table 6: Performance and Energy Metrics for ALBERT Configurations

4.4. Comprehensive Analysis Across Models

The results across BERT, RoBERTa, DistilBERT, and ALBERT demonstrate consistent trade-offs, where compression techniques reduce total energy consumption, CO₂ emissions, CPU and RAM usage, and training time with minor impacts on accuracy, precision, recall, and F1-score. Dynamic Batching emerges as the most balanced approach across all models, often maintaining or even improving performance metrics while achieving significant efficiency gains. For example, in DistilBERT and ALBERT, Dynamic Batching reduces total energy by factors of 7.9 and 6.9, respectively (to 0.00929 kWh and 0.0186 kWh), CO₂ emissions by factors of 16.2 and 3.6 (to 0.00129 kgCO₂e and 0.00497 kgCO₂e), training times to 8.92 and 17.8 minutes, and slightly improves accuracy over baselines (from 0.9260 to 0.9288 for DistilBERT and 0.9266 to 0.9276 for ALBERT), with similar enhancements in precision, recall, and F1-score. This slight improvement may result from more stable gradient updates under dynamic batching.

In BERT and RoBERTa, Dynamic Batching achieves near-baseline performance (accuracy drops of only 0.0048 and 0.0039, respectively) with total energy reductions by factors of 8 and 5.8 (to 0.0178 kWh and 0.0240 kWh), CO₂ emissions by factors of 16.4 and 11.2 (to 0.00247 kgCO₂e and 0.00685 kgCO₂e), proportional decreases in CPU, RAM, and time (e.g., BERT time from 136.5 to 17.1 minutes; RoBERTa from 134.6 to 23.1 minutes), and lower energy per sample. Adapter Modules provide moderate energy reductions by factors of 1.3 to 1.4 (such as 0.1065 kWh for BERT and 0.1051 kWh for RoBERTa) and corresponding decreases in other efficiency metrics, with

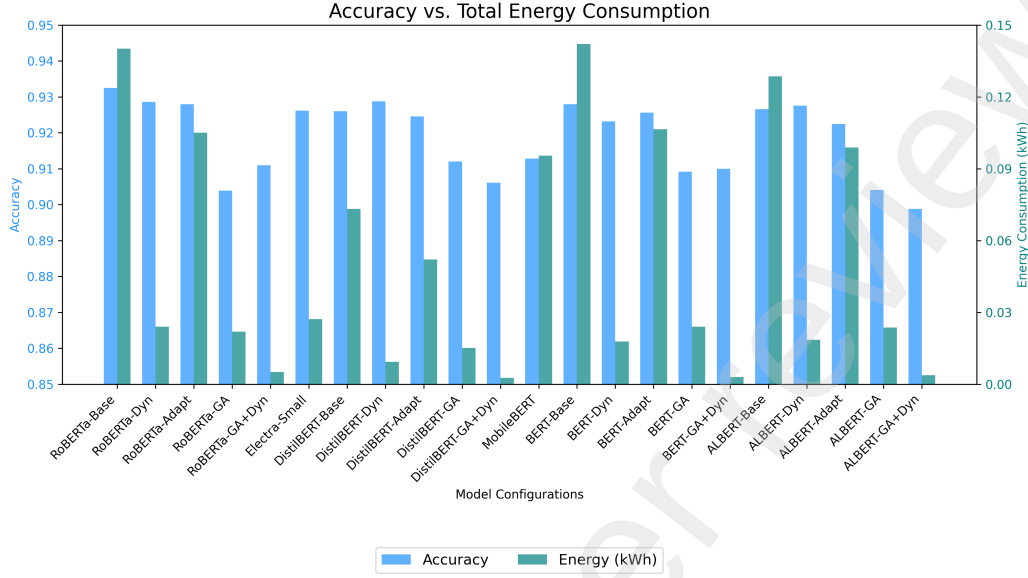


Figure 4: Accuracy vs. Total Energy Consumption for All Model Configurations

very slight performance drops (e.g., accuracy reductions of 0.0024 to 0.0041 across models), making them suitable for scenarios prioritizing parameter efficiency without major sacrifices in precision or recall. However, ALBERT with Adapters shows anomalously high CO₂ emissions (0.0635 kgCO₂e vs. baseline 0.0178 kgCO₂e), likely due to the computational overhead of training additional adapter parameters, which offsets some efficiency gains.

Genetic Algorithm (GA) Optimized offers strong efficiency improvements, with total energy reductions by factors of 4.8 to 6.4 (e.g., to 0.0151 kWh in DistilBERT and 0.0219 kWh in RoBERTa), CO₂ emissions by factors of 2.2 to 25.3 (e.g., to 0.00304 kgCO₂e for RoBERTa), CPU, RAM, time (e.g., 14.5 minutes for DistilBERT), and energy per sample, but at the cost of consistent reductions across all performance metrics (accuracy, precision, recall, F1) in the range of 0.014 to 0.0286. Overall, Dynamic Batching consistently achieves the best trade-offs in both performance and all efficiency metrics, making it a preferred technique for sustainable NLP applications where environmental impact and computational resources are critical considerations.

4.5. Comparison of Optimized Models with Lightweight Baselines

To evaluate the results of the optimized models, we compare the best-balanced configuration for each model (Dynamic Batching, which provides

the optimal trade-off as analyzed in Section 4.3) with the lightweight baselines Electra-small and MobileBERT. BERT-Dynamic (accuracy 0.9232, total energy 0.0178 kWh) achieves lower energy consumption than MobileBERT (accuracy 0.9128, total energy 0.0954 kWh) and comparable to Electra-small (accuracy 0.9262, total energy 0.0272 kWh) while maintaining high accuracy, demonstrating superior efficiency without compromising much on performance.

Metric	Electra-small (Baseline)	MobileBERT (Baseline)
Accuracy	0.9262	0.9128
Precision	0.9262	0.9134
Recall	0.9262	0.9128
F1-Score	0.9262	0.9130
Total Energy (kWh)	0.0272	0.0954
CO ₂ (kgCO ₂ e)	0.0073	0.0613
CPU (kWh)	0.0185	0.0649
RAM (kWh)	0.0087	0.0305
Time (min)	26.1	91.6
Energy/Sample (kWh/sample)	6.70×10^{-7}	2.35×10^{-6}

Table 7: Performance and Energy Metrics for Lightweight Baseline Configurations

RoBERTa-Dynamic (accuracy 0.9286, total energy 0.0240 kWh) outperforms both Electra-small and MobileBERT in accuracy, with energy consumption close to Electra-small and substantially lower than MobileBERT, highlighting its ability to deliver top-tier results at reduced environmental and computational costs. DistilBERT-Dynamic (accuracy 0.9288, total energy 0.00929 kWh) provides the highest accuracy among the optimized models with the lowest energy use, surpassing Electra-small and MobileBERT in both key metrics and underscoring the effectiveness of compression on already efficient architectures.

ALBERT-Dynamic (accuracy 0.9276, total energy 0.0186 kWh) offers competitive accuracy and lower energy than Electra-small, while being significantly more efficient than MobileBERT. These optimized models, after applying compression techniques, demonstrate a more harmonious balance of performance and energy efficiency compared to Electra-small and MobileBERT, which, despite their smaller parameter counts, exhibit higher relative energy consumption or lower accuracy in this evaluation. This proves that targeted compression techniques, particularly Dynamic Batching, yield superior trade-offs for sustainable NLP, enabling high performance with mini-

mized CO₂ emissions, CPU/RAM usage, and training times.

5. Conclusion

This study proposed a framework for sustainable multi-class text classification, evaluated on 50,680 ScienceDaily articles across Health Sciences, Physical Sciences & Technology, and Environmental Sciences. By fine-tuning four Transformer models—BERT, RoBERTa, ALBERT, and DistilBERT—and benchmarking against Electra-small and MobileBERT, we demonstrated that Dynamic Batching, Adapter Modules, and Genetic Algorithms can collectively achieve up to 87.5% energy savings while maintaining competitive accuracy. Energy consumption and carbon emissions were rigorously monitored using CodeCarbon, ensuring a reliable assessment of the environmental footprint of NLP training.

The proposed optimizations directly contribute to the design of power-aware software systems and resource management strategies. For instance, Dynamic Batching enables adaptive control of power-performance trade-offs, making the models suitable for deployment in low-power environments, edge devices, or eco-friendly data centers. Adapter-based fine-tuning further reduces parameter overhead, offering a balance between performance and energy use for large-scale scientific text analytics. These results align with the broader objectives of sustainable computing by minimizing both operational energy costs and carbon emissions.

Beyond NLP, the methodology provides generalizable insights into energy-aware algorithm design for high-performance computing, enabling scalable adoption in green data centers, enterprise computing, and smart control systems. Future work will extend this framework to multilingual corpora and explore advanced compression methods such as quantization and pruning integrated with energy-aware hardware accelerators. Developing middleware and monitoring tools for collective optimization of power and performance will support wider adoption of sustainable machine learning practices, reinforcing the ecological and societal impact of computing.

Supplementary Material

Supplementary material will be provided upon request.

Data Availability

The dataset used in this study will be made available upon reasonable request from the corresponding author after the manuscript is accepted for publication.

Code Availability

The code used for this study will be made available upon reasonable request from the corresponding author after the manuscript is accepted for publication.

References

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, *Attention is all you need*, Advances in Neural Information Processing Systems, 30, 5998–6008, 2017
<https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- Emma Strubell, Ananya Ganesh, Andrew McCallum, *Energy and policy considerations for deep learning in NLP*, Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 3645–3650, 2019,
<https://doi.org/10.18653/v1/P19-1355>.
- David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, Jeff Dean, *Carbon emissions and large neural network training*, arXiv:2104.10350, 2021,
<https://doi.org/10.48550/arXiv.2104.10350>.
- Eileen Paula, Jayesh Soni, Himanshu Upadhyay, Leonel Lagos, *Comparative analysis of model compression techniques for achieving carbon efficient AI*, Scientific Reports, 14, 16259, 2024
<https://doi.org/10.1038/s41598-025-07821-w>.
- Prakhar Ganesh, Yao Chen, Xin Lou, Mohammad Ali Khan, Yin Yang, Hassan Sajjad, Preslav Nakov, Deming Chen, Marianne Winslett, *Compressing large-scale transformer-based models: A case study on BERT*, Transactions of the Association for Computational Linguistics, 9, 1061–1080, 2021, https://doi.org/10.1162/tacl_a_00413.

- Alexandra Sasha Luccioni, Sylvain Vigui er, Anne-Laure Ligozat, *Quantifying the carbon emissions of machine learning*, arXiv:1910.09700, 2019, <https://doi.org/10.48550/arXiv.1910.09700>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, *BERT: Pre-training of deep bidirectional transformers for language understanding*, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 1, 4171–4186, 2019, <https://doi.org/10.18653/v1/N19-1423>.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov, *RoBERTa: A robustly optimized BERT pretraining approach*, arXiv:1907.11692, 2019, <https://doi.org/10.48550/arXiv.1907.11692>.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, Samuel R. Bowman, *GLUE: A multi-task benchmark and analysis platform for natural language understanding*, Proceedings of the 2018 EMNLP Workshop BlackboxNLP, 353–355, 2018, <https://doi.org/10.18653/v1/W18-5446>.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, Radu Soricut, *ALBERT: A lite BERT for self-supervised learning of language representations*, International Conference on Learning Representations, 2020, <https://openreview.net/pdf?id=H1eA7AEtvS>.
- Victor Sanh, Lysandre Debut, Julien Chaumond, Thomas Wolf, *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter*, arXiv:1910.01108, 2019, <https://doi.org/10.48550/arXiv.1910.01108>.
- Kevin Clark, Minh-Thang Luong, Quoc V. Le, Christopher D. Manning, *ELECTRA: Pre-training text encoders as discriminators rather than generators*, International Conference on Learning Representations, 2020, <https://openreview.net/pdf?id=r1xMH1BtvB>.
- Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, Denny Zhou, *MobileBERT: A compact task-agnostic BERT for resource-limited devices*, Proceedings of the 58th Annual Meeting of the Association for

Computational Linguistics, 2158–2170, 2020
<https://doi.org/10.48550/arXiv.2004.02984>.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, Qun Liu, *TinyBERT: Distilling BERT for natural language understanding*, Findings of the Association for Computational Linguistics: EMNLP 2020, 4163–4174, 2020
<https://doi.org/10.48550/arXiv.1909.10351>.

Tianqi Chen, Bing Xu, Chiyuan Zhang, Carlos Guestrin, *Training deep nets with sublinear memory cost*, arXiv:1604.06174, 2016, <https://doi.org/10.48550/arXiv.1604.06174>.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, Sylvain Gelly, *Parameter-efficient transfer learning for NLP*, International Conference on Machine Learning, 2790–2799, 2019
<https://proceedings.mlr.press/v97/houlsby19a.html>.

Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Kamath, Ivan Vulić, Sebastian Ruder, Kyunghyun Cho, Iryna Gurevych, *AdapterHub: A framework for adapting transformers*, Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 46–54, 2020, <https://doi.org/10.18653/v1/2020.emnlp-demos.7>.

Hanrong Ye, Hongyu Wu, Jianfei Liu, Wei Zhang, Yang Sun, *DBS: Dynamic batch size for distributed deep neural network training*, arXiv:2007.11831, 2020, <https://doi.org/10.48550/arXiv.2007.11831>.

Roy Schwartz, Jesse Dodge, Noah A. Smith, Oren Etzioni, *Green AI*, Communications of the ACM, 63(12), 54–63, 2020
<https://doi.org/10.1145/3381831>.

Wu, J., Chen, S., Liu, X., *Efficient hyperparameter optimization through model-based reinforcement learning*, Neurocomputing, 409, 381–393, 2020
<https://doi.org/10.1016/j.neucom.2020.06.064>.

Xiang Zhang, Junbo Zhao, Yann LeCun, *Character-level convolutional networks for text classification*, Advances in Neural Information Processing

Systems, 28, 649–657, 2015

<https://proceedings.neurips.cc/paper/2015/hash/250cf8b51c773f3f8dc8b4be867a9a02-Abstract.html>.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, Luke Zettlemoyer, *BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension*, Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 7871–7880, 2020, <https://doi.org/10.18653/v1/2020.acl-main.703>.

Thierry Tambe, Coleman Hooper, Lillian Pentecost, En-Yu Yang, Marco Donato, Victor Sanh, Alexander M. Rush, David Brooks, Gu-Yeon Wei, *EdgeBERT: Sentence-level energy optimizations for latency-aware multi-task NLP inference*, Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture, 3–18, 2021 <https://doi.org/10.1145/3466752.3480095>.

Courty, B., Schmidt, V., Luccioni, S., Goyal-Kamal, Coutarel, M., Feld, B., Lecourt, J., Connell, L., Saboni, A., Inimaz, supatomic, Leval, M., Blanche, L., Cruveiller, A., ouminasara, Zhao, F., Joshi, A., Bogroff, A., de Lavoreille, H., Laskaris, N., Abati, E., Blank, D., Wang, Z., Catovic, A., Alencon, M., Stechly, M., Bauer, C., de Araujo, L. O. N., JPW, MinervaBooks, *mlco2/codecarbon: v2.4.1*, Zenodo, 2024 <https://doi.org/10.5281/zenodo.11171501>.

Samuli Reinikainen, *Genetic Algorithm for Pruning Transformers*, Master’s Thesis, Tampere University, 2024 <https://trepo.tuni.fi/bitstream/handle/10024/158561/ReinikainenSamuli.pdf?sequence=2>.